


《新生研讨课》报告

# 「昨日圆车」六边形战棋的设计与实现

班 级: _____

姓 名: _____

学 号: _____

教 师: 任　　骜

重庆大学　本科生院

# 昨日圆车（HEX WAGON）——基于六边形网格的双人回合制战棋游戏

作者 1　　作者 2　　作者 3　　作者 4
第 XX 班　第 XX 组

摘要：本文围绕《新生研讨课》的实践要求，设计并实现了一款运行于浏览器、支持双人实时对战与自定义兵种的六边形回合制战棋游戏「昨日圆车」。（1）目标：在尽量少的技术栈内做出一款规则完整、可复现实验、可公网联机、并且允许玩家自己扩展内容的策略游戏，用它检验「规则层与界面层解耦」这一工程方法在「持续迭代」场景下是否依然成立。（2）方法：以轴向坐标对边长可调的六边形地图建模；把放置、替换、射程、光环、大本营结算与断联移除等规则实现为不依赖渲染的纯逻辑引擎，并用 UMD 写法让浏览器与 Node 服务端共用同一份规则代码；界面层用 Canvas 2D 绘制；单人模式采用「五项评估函数 + 迭代加深 α-β 搜索」的启发式 AI；联机采用 WebSocket 传输与服务端权威同步；自建兵种采用「客户端编辑、服务端校验存储、对局按编号下发」的轻量 UGC 方案；新手教学以纯数据描述 7 个关卡。（3）结果：完成本地双人、单人 AI、联机对战、新手教学四种模式，兵种从 3 种扩展到 7 种，并新增棋子替换、可配置光环、攻击范围预览与自建兵种工坊；规则引擎、AI 与联机协议分别通过 678 项、97 项与 44 项自动化断言，合计 819 项；新版 AI 对旧版的对抗成绩为 20 胜 0 负，对随机策略 20 胜 0 负，自对弈平均分胜负的回合数由 96.9 降至 52.3；系统部署于公有云，公网可完成双人实时对局。

关键词：六边形网格；回合制策略游戏；游戏规则引擎；游戏人工智能；WebSocket；用户生成内容

# 1　引言

## 1.1　研究背景

电子游戏是软件工程与人工智能的典型交叉载体。与一般的增删改查系统相比，游戏对「状态一致性」和「规则正确性」的要求要苛刻得多：任何一个判定分支写错，都会直接表现为玩家可见的错误结果，而且很难靠人工穷举发现。在各类游戏中，回合制战棋（strategy wargame）尤其适合作为课程实践的载体——它没有实时操作的压力，规则可以完全形式化，状态空间规模适中，能够在一学期内完成从需求到部署的闭环，同时留出足够的空间去讨论平衡性与算法优化。

从学习角度看，做一个战棋游戏会同时遇到四类问题：几何建模（如何在网格上表示位置、邻接和距离）、规则工程（如何把「同时结算」这类全局判定写对且可测）、算法设计（如何在合理的计算量内做出不犯傻的决策）以及系统集成（如何让两台设备上的游戏状态保持一致）。这四个问题恰好覆盖了程序设计、数据结构、算法与计算机网络的核心内容。而在实际开发中还会出现第五类问题——需求是持续增长的：一款能玩的原型交付之后，玩家（在本项目中就是同学）会不断提出新兵种、新机制、自定义内容的需求。能否在规则不断膨胀的同时保持测试可回归、前后端不失配，是本文后半部分真正关心的事情。

## 1.2　本文研究的具体问题

本文要解决的核心问题是：如何用尽量少的技术栈，实现一款规则完备、可双人实时对战、可以随时调整数值做平衡实验、并且允许玩家自行扩展兵种内容的战棋游戏。围绕这个问题，拆成四个子问题：（1）网格与邻接的表示——如何用最少的状态描述一张边长可调的大六边形地图，并让「只能放在己方连通块边缘」「射程 2 能打到第 2 圈」这类规则只依赖整数运算；（2）回合制结算的一致性——伤害判定是「同时」发生的，死亡之后还要连锁处理失去补给的棋子，如

何保证同一份规则在单机、AI 与联机三种场景下结果完全一致；（3）实时同步——联机时双方的判定绝不能各自为政，如何在弱网、断线与误操作下保证状态收敛；（4）规则的可扩展性——当兵种从 3 种增加到 7 种、并且允许玩家自定义数值与光环时，如何让「服务端权威」不被玩家提交的数据绕过。

## 1.3 国内外研究现状

六边形网格在战棋与策略游戏中被广泛采用，其坐标系统已有成熟总结：Red Blob Games 系统地给出了偏移坐标、立方坐标与轴向坐标三种表示及相互转换，并给出了距离、邻接环、直线与范围查询的通用算法[1]，本文的网格层基本沿用了这套方案。在博弈智能方面，传统做法是「极小极大搜索 + 手工评估函数」，深蓝即以此在 1997 年击败国际象棋世界冠军[2]；此后蒙特卡洛树搜索（MCTS）把随机模拟引入博弈树，显著降低了对评估函数精度的依赖[3]；AlphaZero 进一步用自对弈与深度网络替代了人工特征，在国际象棋、将棋与围棋上达到超人类水平[4]。在实时 Web 方面，IETF 在 RFC 6455 中定义了 WebSocket 协议[5]，为浏览器与服务器之间的全双工通信提供了标准方案，这也是当前在线对战类网页游戏最常用的传输方式。此外，游戏 AI 的工程实践已有系统总结[6][7]，尤其是「感知—决策—行动」的分层结构，对本文的 AI 模块设计有直接参考价值。

需要说明的是，上述研究多针对算力充足、状态空间极大的场景。本文面对的是一个单局 40～120 回合、每回合仅 3 点费用的轻量战棋，其动作空间远小于国际象棋，因此并不需要 MCTS 或神经网络：一个设计良好的评估函数就足以提供可玩的对手，而在此之上加一层受时间预算约束的剪枝搜索还能进一步提高棋力——这一判断在后文 3.3 节的实验数据中被分两步验证了。

## 1.4 本文采用的方法

本文采用「规则层与界面层解耦 + 服务端权威 + 数据驱动扩展」

的总体思路。具体包括五点：（1）把全部规则写进一个无副作用的纯逻辑引擎 engine.js，界面只负责把状态画出来、把点击翻译成动作，规则正确性由引擎的自动化测试来保证；（2）引擎采用 UMD 写法，浏览器通过 script 标签加载、Node 端通过 require 加载，前后端共用同一份代码，从结构上消除「客户端能下、服务端不认」的分歧；（3）联机状态由服务端裁定并广播，客户端只做乐观渲染与断线重连；（4）把所有数值（地图边长、障碍密度、每回合费用、先手方、后手补偿、七种兵种的四项属性、大本营血量）抽成设置项，使平衡性实验不需要改代码；（5）把「兵种」本身抽成数据：内置兵种是数据，玩家自建的兵种也是数据，引擎只认「id + 属性」的字典，因此新增兵种不需要改动规则代码。

### 1.5 主要结果

最终实现的系统包含四种模式：本地双人、单人 AI、联机对战与新手教学，兵种共 7 种（步兵、坦克、炮兵、指挥所、狙击兵、奴隶主、奴隶）。规则引擎、AI 与联机协议分别通过 678 项、97 项与 44 项断言，合计 819 项，全部在一次命令内跑完；新版 AI 对旧版 AI 的对抗成绩为 20 胜 0 负，对随机策略同为 20 胜 0 负；系统以静态资源 +Node 服务端的形式部署于公有云，公网可直接进行双人实时对局。此外还实现了三项扩展能力：棋子替换（把已放置的己方棋子原地换成别的兵种）、可配置光环（范围 1～3 格、四种效果、同类取最高不叠加）与自建兵种工坊（在浏览器内设计兵种、上传到服务器、由其他玩家在设置中选用）。

## 2 文献综述

### 2.1 六边形网格与战棋规则建模

六边形网格相对正方形网格的最大优势在于「邻接距离更均匀」：正方形四邻域的欧氏距离并不相等，斜向移动会更快，而六边形每个

格子到六个邻居的距离完全一致，这使得射程、移动力这类以「步数」计量的规则更自然。Red Blob Games 的综述指出，六边形网格常用的表示有三种：偏移坐标（offset）、立方坐标（cube）与轴向坐标（axial）[1]。其中立方坐标用三个整数 $(x, y, z)$ 且恒有 $x + y + z = 0$，几何意义最直观，距离可以用 $\max(|\Delta x|, |\Delta y|, |\Delta z|)$ 直接计算；轴向坐标相当于把立方坐标中的一个分量省掉，只用 $(q, r)$ 两个整数即可唯一确定格子，是存储效率与计算便利性的折中。本文选择了轴向坐标，原因是它既避免了偏移坐标在奇偶行之间的分支判断，又比立方坐标少存一个冗余分量。

在规则建模方面，教科书式的做法是把「地图、棋子、规则」三者分离：地图只描述哪些格子可通行，棋子只描述属性与归属，规则则以查询的方式在地图与棋子上做判定[6]。这种分离的最大好处是规则可以被单独测试。以本文的「放置合法性」为例，规则要求棋子只能放在「与己方大本营连通的那片棋子」的边缘，这本质上是一次图的广度优先搜索；而「射程判定」则退化为两次坐标差的整数运算。二者都不涉及像素坐标，因此可以在没有渲染环境的命令行里被完整验证。Red Blob Games 还特别讨论了地图对称性的问题：如果障碍物是随机生成的，就必须保证双方地形等价，否则先手优势会被地形放大。这一点直接影响了本文的设计——地图障碍采用中心对称成对生成[1]。

## 2.2 博弈树搜索与评估函数

棋类 AI 的经典框架是极小极大搜索配合局面评估函数。深蓝的成功说明了两个要素的重要性：足够深的搜索与高质量的评估函数[2]。当状态空间继续增大时，穷举搜索不再可行，蒙特卡洛树搜索通过随机模拟与置信上界选择来平衡「探索」与「利用」，在围棋等评估困难的博弈上取得了突破[3]；Browne 等人的综述系统总结了 MCTS 的四个阶段（选择、扩展、模拟、回溯）及其各种变体[3]。AlphaZero 则进一步把策略网络与价值网络引入 MCTS，用自对弈替

代人类棋谱，在国际象棋、将棋与围棋上均超过当时的专用程序[4]。

对于本文这样的小体量战棋，直接套用 MCTS 并不划算：单回合的可行动作数在几十到上百量级（每个合法落点、每种兵种、狙击兵还要乘六个朝向、以及是否结算的任意组合），模拟一次完整对局需要数百次状态转移，在浏览器主线程上会给交互带来明显卡顿。因此本文选择了更「轻」的方案：以评估函数为核心，配合剪枝搜索，并把搜索深度限制在时间预算之内。Buckland 在《Programming Game AI by Example》中把这种结构称为「感知—决策—行动」的简化版，并指出在动作空间有限时，评估函数的工程质量比搜索深度更影响最终表现[6]，本文后续的版本迭代（3.3 节）正好验证了这一点：仅把评估函数从「按兵种硬编码」改为「按实际攻防 + 光环边际收益 + 中央加成 + 吃子收益」计算，对旧版就取得 7 胜 2 负 11 平；在此基础上再把一步前瞻换成迭代加深 α-β，对同一基线 20 局全胜，且自对弈平均分胜负的回合数从 96.9 降到 52.3。

### 2.3 实时联机与浏览器渲染

网页端的实时对战有两条传统路线：轮询（polling）与长连接。前者实现简单但延迟高、开销大；后者以 WebSocket 为代表，通过一次 HTTP 升级握手建立全双工通道，之后双方可直接收发消息帧[5]。RFC 6455 定义了握手流程、帧结构与关闭语义，并在协议层内置了 ping/pong 心跳机制，这正是本文联机模块的基础。在架构上，实时对战通常采用「服务端权威」（server-authoritative）模型：客户端只发送意图，服务端校验并广播结果，从而避免客户端篡改与两端状态分歧。Yannakakis 与 Togelius 在《Artificial Intelligence and Games》中把这种结构归纳为「游戏状态由权威方持有、玩家输入作为事件流」的模式，并指出它对反作弊与回放的支持最好[7]。

渲染方面，浏览器提供了 DOM、SVG 与 Canvas 三种绘制手段。对于本文这种「几十个六边形 + 若干棋子 + 动画特效」的场景，Canvas 2D 的优势在于一次绘制调用即可完成整帧重绘，且易于实现

虚线高亮、渐变与飘字等效果；代价是需要自己处理设备像素比（devicePixelRatio）、窗口缩放与命中检测。实践中我们遇到一个典型问题：在 CSS 弹性布局下若直接按容器尺寸设置 canvas 的 width/height 属性，会触发「设置尺寸 → 容器变化 → 再次 resize」的反馈循环，最终表现为画布不断放大；解决办法是在读取尺寸之前先把画布清零。这类工程细节通常不会出现在论文中，但对可用性影响很大，故在 3.2.4 节一并记录。

## 2.4 用户生成内容与可扩展设计

让玩家自己创造内容是延长小体量游戏生命周期的常见做法。工程上的关键不在于编辑器本身，而在于「玩家提交的数据如何安全地进入一个由服务端裁定的对局」。Fielding 的 REST 架构风格[8]给出了一条简单可行的路径：内容以资源的形式存在（POST 创建、GET 列表、DELETE 删除），服务端对每一份提交做校验与归一化，客户端只持有资源的编号；对局开始时，客户端提交的是「启用了哪些编号」，服务端再按编号从自己的库中取出定义并注入对局。这样即使玩家在前端修改了数值，也无法让服务端认可一份被篡改的兵种——规则数据始终由权威方持有。这一思路与 2.3 节的「服务端权威」是同一条原则在内容层面的延伸。

与之配套的还有两个工程问题：一是配额，开放上传的接口如果没有限制，很容易被脚本灌满存储，本文采用「按浏览器身份计数的免费额度 + 可用于解除限制的秘钥」这一轻量方案；二是归属，删除一份内容必须能证明「这是你创建的」，本文的做法是创建时返回一枚仅作者可见的删除凭据，后续删除请求必须携带它。这两点都不复杂，但缺了任何一点，开放上传的功能都会在真实环境中迅速失效。

# 3 研究内容

## 3.1 系统架构设计

系统采用四层结构，自上而下分别为表示层、应用层、规则层与传输层，如图 1 所示。层与层之间是单向依赖：上层可以调用下层的接口，下层不感知上层的存在。这样做的直接收益是规则层可以在完全脱离浏览器的环境（Node.js）中被测试，也能被服务端直接复用。




图 1　系统分层架构与同构复用关系

表示层由两个页面组成：开始菜单页（index.html + menu.js）负责模式选择、参数设置、地图预览与自建兵种编辑；对局页（game.html + main.js）负责把引擎状态画出来，并把点击事件翻译成动作。绘制逻辑集中在 renderer.js 中，它只接收「当前应当显示什么」的描述对象，不持有任何游戏状态。应用层包含四个相对独立的模块：ai.js 负责决策，net.js 负责与服务端通信，tutorial.js 以纯数据的形式描述教学关卡，mods.js 负责自建兵种的归一化、本地存储与服务器通信。规则层是系统的核心，hex.js 负责几何运算，engine.js 负责状态与回合，maps.js 与 settings.js 负责地图与参数。传输层的 server.js 同时承担静态托管、WebSocket 房间管理与自建兵种的 REST 接口。

这种分层带来两个具体好处。第一，规则的可测性：由于 engine.js 不触碰 DOM，测试脚本可以直接在 Node 中构造局面、执行动作并断言结果，本文的 678 项规则断言全部以这种方式运行，执行时间

不到 2 秒。第二，联机的一致性：服务端 require 的是同一个 engine.js，因此「客户端认为合法」与「服务端认为合法」在代码层面就是同一个判断函数，不存在两份规则逐渐漂移的风险。这一点在后续不断加入新兵种与新机制时体现得尤其明显——每一条新规则只需要在引擎里实现一次，单机、AI 与联机三条路径同时生效。

截至定稿，系统共有约 7800 行源码（不含测试），其中规则层约 1600 行、表示层约 3000 行、服务端约 670 行；三套测试脚本合计约 1430 行。

## 3.2 技术方案

### 3.2.1 六边形地图与坐标系统

地图是一张由小六边形拼成的大六边形，边长 N 可调（3～8）。格子用轴向坐标 (q, r) 表示，其像素位置由 $x = size \cdot 1.5 \cdot q$、$y = size \cdot \sqrt{3} \cdot (r + q/2)$ 唯一确定，无需为每个格子缓存坐标。地图范围由三个不等式共同给出：$|q| \leqslant N-1$、$|r| \leqslant N-1$、$|q+r| \leqslant N-1$，格数恰为 $3N(N-1)+1$，N=5 时为 61 格。六个邻居的方向向量固定为 $(\pm1,0)$、$(0,\pm1)$、$(+1,-1)$、$(-1,+1)$，两点距离用立方坐标范数 $d=(|\Delta q| + |\Delta r| + |\Delta q+\Delta r|)/2$ 计算，全部是整数运算，如图 2 所示。

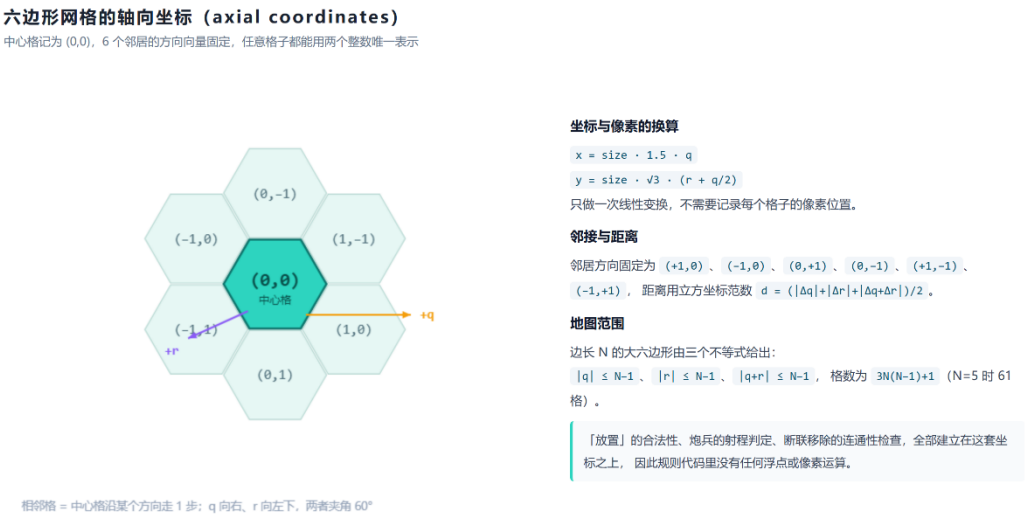



图 2 轴向坐标、邻居方向与棋盘范围

两个大本营固定在地图的上下顶点，坐标分别为 $(0, -(N-1))$ 与 $(0, N-1)$。障碍物的生成采用中心对称：先在候选格中随机抽取一个格子放置障碍，同时把它的中心对称点 $(-q, -r)$ 一起标为障碍，从而保证双方地形严格等价、且两端大本营互为镜像。生成过程还包含两道校验：大本营及其邻格不允许出现障碍；生成后对双方的连通块做一次广度优先搜索，若某一方无法从大本营出发建立连通块则整张图重新生成。此外，本文还提供了 8 张固定种子的「默认地图」，只存「密度 + 种子」两个数字，由同一套生成器确定性地产出，因此同一张图在任何设备上完全一致。

### 3.2.2 规则引擎

引擎的核心状态包括：地图格子、棋子集合（以格子为键的映射）、当前行动方、当前回合数、当前费用、双方的大本营血量以及击杀与损失统计。一次完整的「放置—结算—结束回合」流程如下。

放置的合法性判断依次为：对局未结束、轮到该方、费用足够、目标格在地图内、不是障碍、满足该兵种的额外限制（例如奴隶必须挨着奴隶主）、且属于「己方连通块的边缘」。最后一条的实现是对该方全部棋子做一次广度优先搜索，把与自身大本营连通的集合求出，再取其所有邻居格作为合法落点集合。开局时双方各只有大本营，因此第一回合恰好只有 3 个合法落点。

本版新增了「替换」规则：玩家可以点击自己已经放好的棋子将其选中，再点击下方的兵种卡，把这一格原地换成新的兵种。替换只扣除新棋子的费用，被换掉的棋子在规则上直接消失，不计入击杀也不退还费用；大本营与敌方棋子不能被替换，同兵种原地替换会被拒绝。实现上，替换并没有新增一套流程，而是把「目标格已有己方棋子」从非法条件改成了合法分支：canPlace 在检测到占位者是己方非基地棋子时返回该棋子对象，place 则在落子前把它优雅地移除。这样单机、AI 与联机三条路径无需任何改动即可支持替换。

结算是本系统最需要小心的一步。它分为三个阶段：第一阶段计

算「伤害图」，对场上每一个棋子（含大本营），遍历所有敌方棋子，若二者距离不大于攻击方的射程（且该攻击方有攻击力），则把攻击方的实际攻击力累加到该棋子身上；第二阶段按「血量 ≤ 攻击之和」同时判定死亡，由于用的是结算前的局面，双方完全可能在同一轮同时损失棋子；第三阶段处理补给线，把死亡棋子移除后，对双方重新做连通性搜索，与己方大本营失去连接的棋子一并移除。三个阶段都必须基于同一份快照，否则会出现「先死的一方少挨一刀」这类顺序依赖。结算不消耗费用、不结束回合，回合结束前可以反复执行，这也是本游戏的核心策略点之一。

为了让联机与回放成为可能，引擎实现了完整的序列化：toJSON 输出纯数据结构，fromJSON 逐字段重建，clone 则用于 AI 的候选动作模拟。所有状态（包括击杀统计、单次结算的最大击杀数、狙击兵的朝向、本回合已召唤的奴隶数）都会被序列化，因此服务端广播的状态与客户端本地的状态在结构上完全一致。

### 3.2.3 兵种体系与光环

系统共有 7 种兵种，其中 6 种可以用费用购买，奴隶只能由奴隶主召唤。各兵种的定位与数值如表 1 所示（数值均可在设置中调整）。

| 兵种 | 血量 | 攻击 | 射程 | 费用 | 特点 |
|---|---|---|---|---|---|
| 步兵 | 3 | 1 | 1 | 1 | 便宜、耐打，用来铺路与占格子 |
| 坦克 | 4 | 1 | 1 | 2 | 血量最高的常规单位，适合顶在前排 |
| 炮兵 | 3 | 1 | 2 | 2 | 射程 2，可以隔一格输 |

| 兵种 | 血量 | 攻击 | 射程 | 费用 | 特点 |
| --- | --- | --- | --- | --- | --- |
| | | | | | 出 |
| 指挥所 | 2 | 0 | 0 | 2 | 自身不能攻击，为周围一格内的己方棋子加攻 |
| 狙击兵 | 2 | 1 | 3（直线） | 2 | 只打自己朝向的一条直线，落子时选方向 |
| 奴隶主 | 3 | 1 | 1 | 2 | 每回合免费召唤 1 个奴隶 |
| 奴隶 | 2 | 1 | 1 | 免费 | 只能由奴隶主召唤，且必须落在奴隶主周围一圈 |

其中四种兵种带有特殊机制。指挥所自身不能攻击，但会向周围一格内的己方棋子提供攻击力加成，是典型的「阵型型」兵种；狙击兵只能攻击自己朝向的一条直线，落子时必须先选方向，方向在之后的回合中不会改变，因此它的价值高度依赖于放置位置；奴隶主每回合可以免费召唤一个奴隶，被召唤出的奴隶相当于「不花费用的一次落子」，是抢占格子与消耗对方伤害的手段；奴隶则是纯消耗品，血量低但免费。

光环是本版最重要的机制抽象。最初的实现里「指挥所加成」是硬编码的特例（if 该棋子是指挥所则相邻友军 +1），在加入自建兵种后，这种写法无法支持玩家自定义光环，于是把它抽象成数据：每种

兵种可以携带一个光环对象，包含三个字段——范围（1～3 格）、效果（己方加攻 / 己方加血 / 敌方减攻 / 敌方减血）与数值（1～5 点）。计算任意棋子的实际属性时，先收集所有覆盖它的光环，再按效果分别取其中的最大值：

实际攻击力 = 基础攻击 + max(覆盖它的己方加攻光环) − max(覆盖它的敌方减攻光环)；实际血量 = 基础血量 + max(覆盖它的己方加血光环) − max(覆盖它的敌方减血光环)，且不低于 1。

「取最大值而不是求和」是经过一轮实测后确定的规则。若允许叠加，多个同类型光环会呈线性膨胀（两个指挥所把步兵的攻击从 1 提到 3），阵型一旦成型便难以撼动，对局会迅速退化为「谁先把指挥所堆起来」；而取最大值时，第二个同类光环只对「尚未被覆盖的单位」有边际收益，这让光环兵种成为一个需要位置感的资源，而不是可以无脑叠加的数值。这个例子也说明了数据驱动的另一个好处：规则改动只需要动一个函数（auraSum 改为 auraMax），7 个兵种与所有自建兵种同时获得新规则。

#### 3.2.4 渲染与交互

渲染层使用 Canvas 2D，每一帧按「地板与障碍—合法落点—攻击范围预览—棋子—光环与射线—血条—飘字特效」的顺序绘制。为了保证高分屏不模糊，画布的实际像素尺寸按 devicePixelRatio 放大，再用样式尺寸缩回逻辑尺寸；窗口尺寸变化时先清空画布再测量容器，避免「测量—设置—再测量」的反馈循环。落点高亮使用虚线描边并在费用不足时改为红色，棋子被结算击杀时在格子上飘出「已造成/需要」的数字（例如 3/4 表示已吃 3 点、离致死还差 1 点），这一细节显著降低了新手理解结算规则的成本。

本版新增了「攻击范围预览」，它是把规则可视化的一次尝试：玩家选好兵种后，把指针停在任意合法落点上，引擎会用一个「假想棋子」调用同一套射程判定（attackCellsFrom），得出「如果放在这里能打到哪些格子」，再用金色虚线格与中心圆点画出来；如果范围内

正好有敌方棋子，还会在其上套一个红圈。这个功能的实现只用了不到三十行代码，因为它复用了引擎里已有的射程函数——规则一旦被抽成纯函数，可视化就变成了「换个地方再调用一次」。

交互层把鼠标与触摸事件统一为「点击某格」的语义，再交给动作分发函数。这里踩过一个交互设计的坑：最初「替换」采用的是「先选兵种、再点棋子」，结果玩家在已选中兵种的状态下点击自己的棋子，会意外触发替换而不是查看信息。改为「先点棋子选中它，再点下方兵种卡」之后，选中态有了明确的视觉反馈（该格加一圈琥珀色虚线边框、详细卡显示「已选中，点下面任意兵种卡替换」），误操作随之消失。这条经验被写进了设计原则：破坏性操作用「先选目标、再选手段」，可逆操作用「先选手段、再选目标」。

### 3.2.5 人工智能设计

AI 采用「评估函数 + 迭代加深 α-β 搜索」。评估函数经过三轮重构，从「按兵种硬编码」改为「按实际状态计算」，现在的五项是：（1）棋子价值：以实际攻击力与实际血量为基数，并把该棋子光环的「边际覆盖」折算进去——即它能为多少尚未被同类光环覆盖的单位提供多少点加成；（2）中央加成：按棋子到棋盘中心的六边形距离给出 0～1 的权重，再按射程放大，越靠中央、射程越长的棋子估值越高（炮兵置于中央比置于外圈高出约 28%），理由是中央位置能同时压住三条方向；（3）吃子估值：先算一次伤害图，把「本回合结算就会死」的棋子直接按已死处理，并对击杀方额外计入 0.55 倍的敌方损失，于是「能不能吃掉对方」成为分值差里的主项；（4）推进度与火力覆盖：越靠近对方大本营、打击格覆盖到的敌方单位与基地越多，分值越高；（5）子力数量：场上每多一个存活棋子计 +0.9 分，用于鼓励交换与避免无谓损失。

这里有一处值得记录的删减：早期版本里还有一项「残血折价」，按当前血量占最大血量的比例线性打折。重新审阅规则代码后发现这一项是空转——结算只判断「伤害 ≥ 血量」决定生死，从不对存活

棋子扣血，因此场上不存在「残血」状态，大本营也只有「被一击打爆」与「安然无恙」两种情形。这一项被删除，权重让给了吃子与位置。

决策不再是一步前瞻，而是在「原子动作」上做带剪枝的深度搜索。一个回合被拆成四类原子动作：放置、召唤奴隶、结算、结束回合；搜索时对每个原子动作克隆局面并执行，再交给评估函数打分。节点的展开顺序由启发式排序决定——能直接吃掉对方棋子的落点排在前面，其次是能压到对方大本营的落点，结束回合永远排在最后。在此之上做 α-β 剪枝，并按迭代加深的方式逐层加宽：从 1 层开始，每轮把上一轮的最优解提到队首，直到时间预算用完或触达深度上限。实测在边长 7 的中局，1.2 秒预算可以搜索到第 7 层、约 1.8 万节点；超时立即返回上一层的最优解，因此不会出现「想太久卡住界面」的情况。

为了让人能看清对手在想什么，AI 的节奏被显式设计过：每个回合的第一个动作之前先「思考」1～3 秒（这段时间就是搜索预算来源），同一回合的后续动作间隔缩短到约 0.4 秒，界面底部同步显示「电脑正在思考」。搜索是同步的，会占住主线程，因此「思考中」的提示先渲染、再开始计算，否则提示永远画不出来。

AI 还被用作平衡性实验的平台：让两个参数相同的 AI 互相对弈并统计回合数与兵种购买分布，可以粗略反映数值设计的健康程度。最新的统计显示，在默认数值下 AI 几乎不购买坦克——坦克（4 血、2 费）与炮兵（3 血、2 费、射程 2）同价，但在「能站远一格输出」这件事上炮兵优势明显，这是数值层面的问题而非 AI 缺陷，也提示后续应当对坦克做小幅加强。

#### 3.2.6 联机与状态一致性

联机采用房间制：创建者获得一个 4 位数字房号与一枚座位令牌，加入者凭房号进入另一个座位，两人同时在线即自动开局。传输层使用 WebSocket，服务端每 25 秒发起一次 ping 以穿透中间设备；

客户端每 45 秒发送一次应用层心跳，既用于保活，也让云服务器上的进程守护与平台健康检查能够判定实例状态。断线后客户端凭令牌自动重连并回到原座位，服务端为每个座位保留 10 分钟的重连窗口；若对局已结束且双方都离开，房间立即回收。

所有游戏动作（放置、替换、召唤奴隶、结算、结束回合、认输、求和）都先发送到服务端，服务端用同一份 engine.js 校验后执行，再把新状态与事件广播给双方。客户端收到状态后整体替换本地状态而不是增量修改，从根上避免了「增量丢失导致两端分叉」。非法动作（例如不是自己回合、费用不足、狙击兵没带方向）会被服务端拒绝并回传错误提示。求和是唯一允许在非自己回合应答的动作，因此被单独提前处理。

自建兵种在联机中的传递方式值得一提：客户端建房时只下发启用的兵种「编号」列表，服务端按编号从自己的库里取出定义、经过数值夹取后注入对局，再随状态一起广播。这样一来，即使某一方在前端把自建兵种的攻击力改成 100，服务端也不会接受——因为它根本没有读客户端发来的数值。

### 3.2.7 自建兵种工坊

工坊允许玩家在浏览器里设计兵种并发布给其他人使用，界面如图 7 所示。可编辑的字段包括名称、单字图标、简介、血量、攻击、射程、费用、攻击方式（周围一圈 / 一条直线 / 不能攻击）以及光环（范围、效果、数值）。右侧是一张实时预览的兵种卡，玩家改动任何一项，卡片与提示文字都会立即刷新；编辑器还会给出针对性的提示，例如「费用为 0 意味着每回合可以免费铺，注意别太强」「选择直线攻击时部署需要先选方向」。

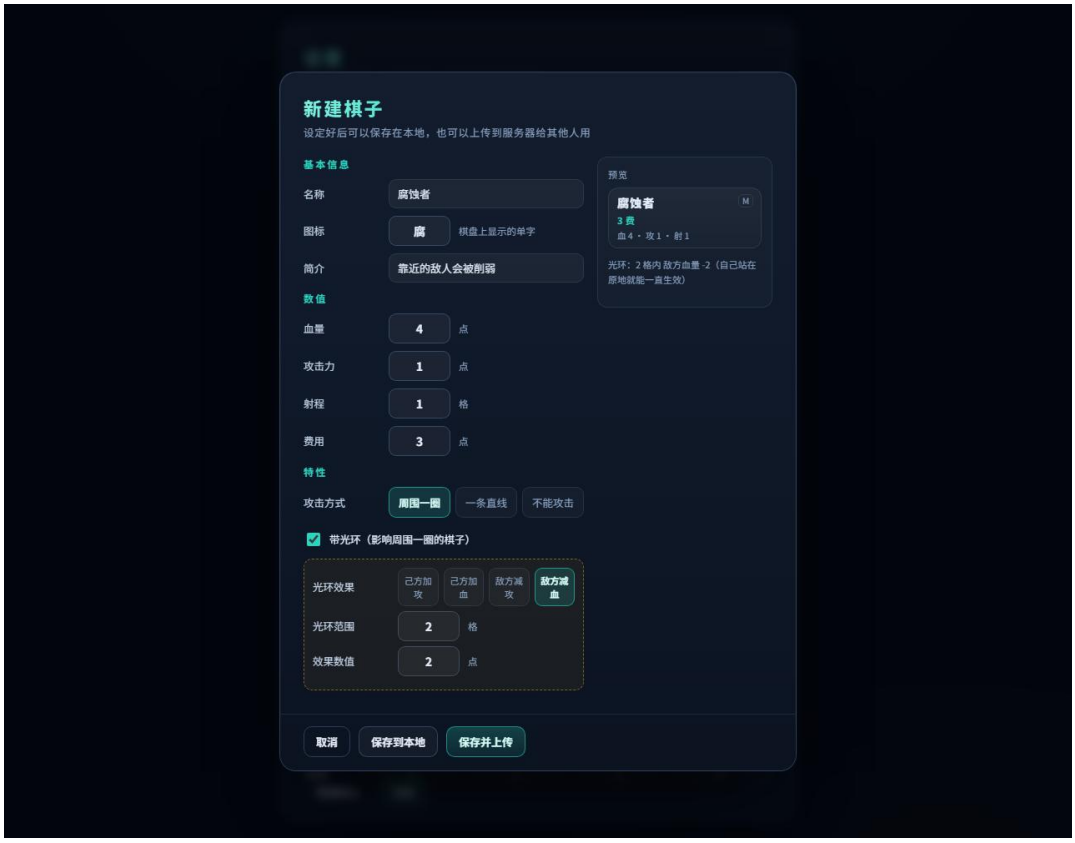



图 7　自建兵种编辑器（左侧表单、右侧实时预览、底部额度与秘钥）

整个流程分为三段。第一段是编辑与本地保存：玩家可以把设计存进浏览器本地存储（localStorage），单机立刻可用，不依赖服务器。第二段是上传：POST /api/mods 提交兵种定义，服务端对其做严格的归一化——名称截断到 10 字、简介 120 字、血量夹在 1～30、攻击 0～20、射程 0～6、费用 0～30、光环范围 1～3 且数值 1～5，并强制「攻击力为 0 则射程必须为 0」这类一致性约束；通过校验后分配一个 mod_xxxxxxxx 编号与一枚仅作者可见的删除凭据，写入服务端的 data/mods.json。第三段是选用：其他玩家在设置里看到服务器上的兵种列表，勾选后即进入自己的商店，一局最多启用 8 个。

为了让这个开放接口在真实环境中可用，加了两条必要的约束。其一是配额：每个浏览器身份在不填写秘钥的情况下最多上传 4 个兵种，填写约定的秘钥则不限制；额度与身份都记录在服务端，前端改不了。其二是归属：删除请求必须携带创建时返回的凭据，否则返回 403，因此别人无法删掉你的兵种。此外列表接口不会返回凭据与

上传者身份。

开发过程中还修正了两个与「删除」有关的问题，值得一提。一是删除后条目仍然显示在列表里：列表的数据源是「本地定义 + 服务器定义缓存」的并集，删除时只清了本地定义，服务器缓存里的那一条立刻又被合并回来；修正办法是删除时同步清空缓存并重新拉取列表。二是删除按钮此前只在「本地有记录」的条目上出现，因此如果玩家换了浏览器，他自己上传的兵种就无法删除——这属于设计上的取舍（服务端无法确认新设备就是原作者），最终的方案是保留凭据、由客户端保存，并在文档中说明。

联机时自建兵种的传递已在 3.2.6 节说明：只传编号，定义由服务端提供。这也意味着对手无需预先安装任何东西就能看到并使用你设计的兵种——房间状态里已经带着它的完整定义。

#### 3.2.8 新手教学与可配置性

新手教学由 7 个预设小局面组成，每关只教一个知识点：放置棋子、费用限制、坦克的血量优势、结算与伤害、炮兵的射程、断联移除、拆掉大本营。每关以纯数据描述：地图尺寸、初始费用、需要预置的棋子（通过引擎的 spawn 接口无视放置规则直接落子）、可用兵种，以及若干「步骤」，每个步骤包含一句提示和一个判定函数。控制器在每次动作后依次检查当前步骤是否完成，完成后自动推进；若玩家做出了与本关目标无关的操作，关卡会自动重置并给出提示。这种「数据 + 判定函数」的写法让关卡可以在 Node 中被完整测试，也便于后续继续扩充。

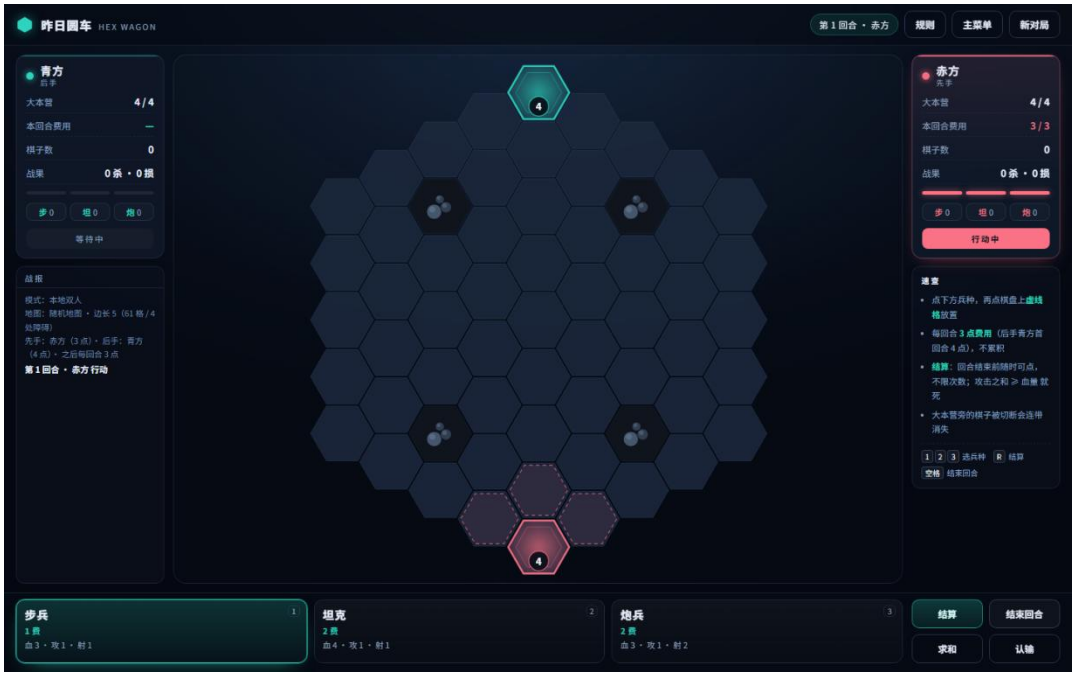
可配置性是本项目的另一条主线。地图来源（随机 / 8 张默认地图）、边长、障碍密度、每回合费用、先手方、后手补偿、七种兵种的属性、大本营血量都可以在设置中修改，并保存在浏览器本地。设置一旦改变，菜单摘要会即时更新；联机时这些参数由建房的玩家一次性下发给服务端，服务端据此创建对局，双方看到的地图与规则完全一致。

## 3.3　项目结果与展示

系统最终拆分为两个页面：开始菜单页（图 3）与对局页（图 4）。菜单页提供四种模式的入口、房号快速加入、服务器状态与当前房间数；对局页左侧为双方状态面板与战报，中间为棋盘，右侧为速查与教学面板，底部为兵种商店与操作按钮。




图 3　开始菜单页（四种模式、快速加入、服务器状态）




图

4　对局页（棋盘、双方状态、战报与兵种商店）

图 5 展示了「选择落点时预览攻击范围」的效果：选中炮兵后把指针停在一个合法落点上，金色虚线格即为该兵种放在此处能覆盖的全部格子，范围内的敌方棋子会被套上红圈；图 6 则是「替换」的交互状态——点击自己的步兵后，该格出现琥珀色虚线边框，左下详细卡显示当前的实际攻防（括号内为光环加成）并提示「已选中这个棋子，点下面任意兵种卡即可替换」，同时底部所有兵种卡都亮起琥珀描边。

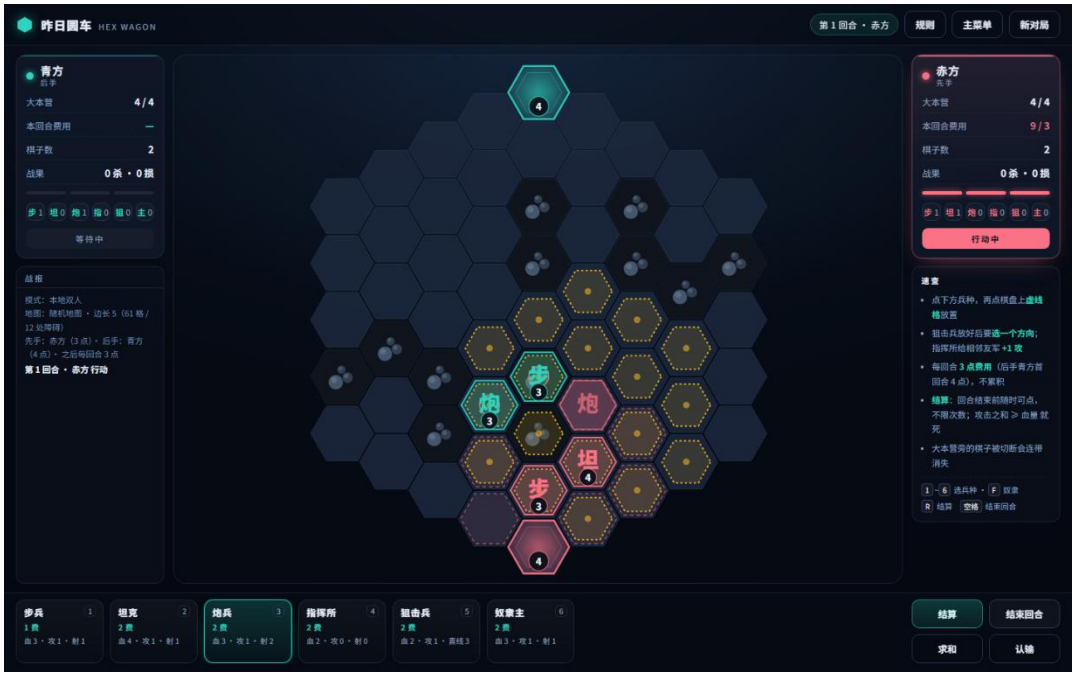



图 5　攻击范围预览（金色虚线为覆盖范围，红圈标记可打击的敌人）

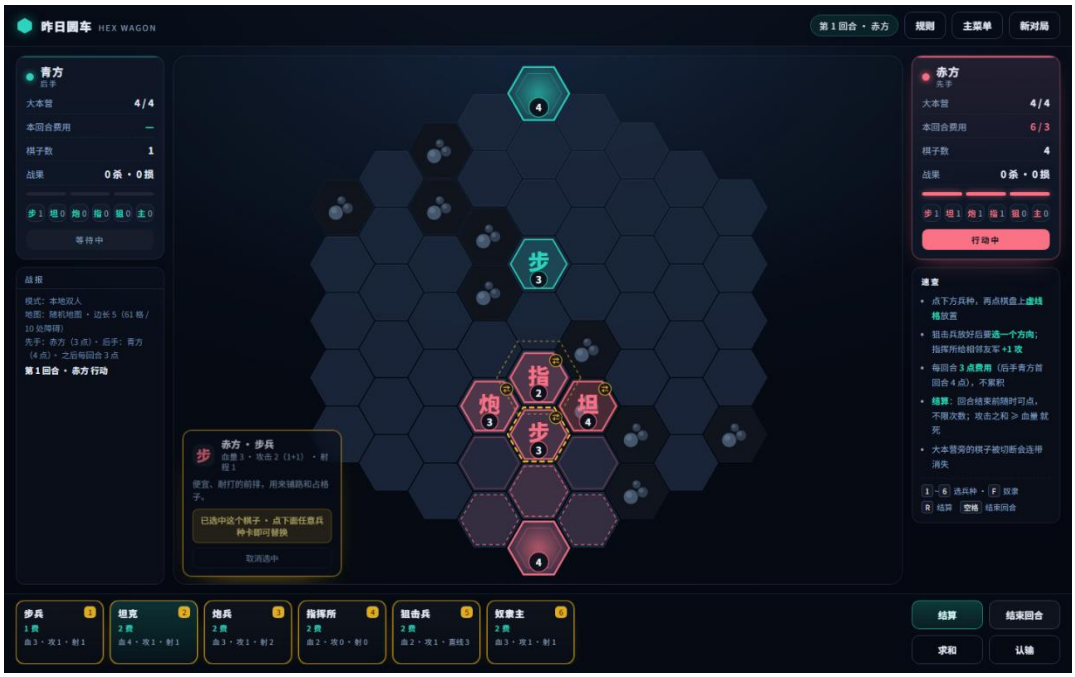



图 6　棋子替换（选中己方棋子后点击兵种卡即可原地替换）

图 8 展示了奴隶的放置限制：奴隶只能落在奴隶主周围一圈，因此选中「免费奴隶」卡片时，高亮的合法落点会收窄为奴隶主相邻的那几格。图 9 则是新手教学中的一关，教学中会屏蔽与当前关卡无关的兵种，并在底部只保留必要的操作按钮。

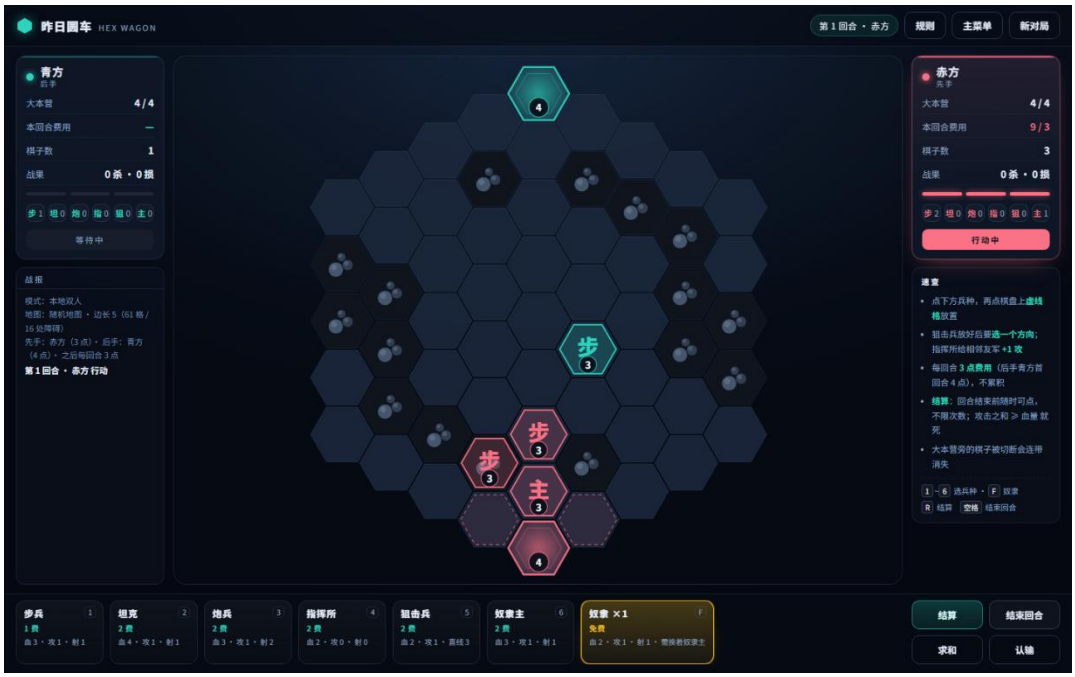



图 8　奴隶只能放在奴隶主周围一圈



图 9　新手教学（坦克扛伤害）

对局结束时给出本局的对比战绩，包括双方击杀、损失、剩余棋子与单次结算的最大击杀数，并统计总回合数、耗时与地图规模。这些数据既是对局复盘材料，也为后续的平衡性调整提供了依据。

系统的功能与自动化测试情况如表 2 所示。除功能测试外，本文还做了三组对抗实验：其一，让 AI 与随机策略对弈 20 局，AI 全部获胜；其二，把改进拆成两步分别与版本控制里保留的上一版 AI 各执先手对弈——第一步只重写评估函数（加入吃子与中央项，搜索深度不变），20 局取得 7 胜 2 负 11 平；第二步再换上迭代加深 α-β 搜索，对同一基线 20 局全胜（20:0）；其三，让新旧两版各自自对弈 12 局，旧版只有 3 局能在回合上限内分出胜负、平均 96.9 回合，新版有 9 局分出胜负、平均 52.3 回合。第一、二组数据说明评估函数的质量是小体量战棋里最划算的改动；第三组数据说明加深搜索主要改善的是「敢不敢换子」——它让对局从互相造兵的对耗变成主动交换，收敛所需的回合数几乎减半。

| 功能模块 | 实现情况 | 验证方式 / 结果 |
| --- | --- | --- |
| 六边形地图（边长 3~8） | 轴向坐标、中心对称障碍、 | 边界与对称性断言全部通过 |

| 功能模块 | 实现情况 | 验证方式 / 结果 |
| --- | --- | --- |
| | 8 张固定默认图 | |
| 七种兵种与费用 | 步兵 / 坦克 / 炮兵 / 指挥所 / 狙击兵 / 奴隶主 / 奴隶 | 数值可调，678 项规则断言通过 |
| 放置与替换 | 连通块边缘 + 额外限制（奴隶需邻接奴隶主）；支持原地替换 | 含边界、非法落点与替换用例 |
| 结算与断联移除 | 同时判定、三阶段处理、可重复结算 | 含同时死亡与连锁移除用例 |
| 光环系统 | 范围 1~3、四种效果、同类取最高不叠加 | 含叠加互斥与不同效果并存的用例 |
| 单人 AI | 五项评估（含中央加成）+ 迭代加深 α - β（4~7 层） | 对随机 20:0；对上一版 20:0；97 项断言 |
| 联机对战 | WebSocket + 服务端权威 + 断线重连 + 自建兵种按编号下发 | 44 项协议与接口断言通过 |
| 自建兵种工坊 | 浏览器内编辑、本地保存、上传（限额 4 个 / 秘钥解除）、按凭据删除 | 接口断言通过，含配额与鉴权用例 |
| 新手教学 | 7 关数据驱动关卡 | 每关目标与预置局面均有断言 |
| 部署 | 静态资源 + Node 服务端 + 进程守护 | 公网双人实时对局可用 |

# 4 结论

本文完成了「昨日圆车」这款六边形双人回合制战棋游戏的设计与实现，主要工作可以归纳为四点。第一，在几何层面选用轴向坐标描述六边形网格，把放置合法性、射程判定与补给连通性全部归结为整数运算与图搜索，使规则代码不依赖任何渲染环境。第二，在工程

层面采用「规则层与界面层解耦 +UMD 同构复用」的方案，让浏览器与服务端共享同一份规则代码，从结构上消除了联机场景下的判定分歧，并使规则可以用 678 项断言在 2 秒内完成回归。第三，在玩法层面把「兵种」「光环」「数值」全部抽成数据，使兵种数量从 3 种扩展到 7 种、并支持玩家自建兵种，而规则代码几乎不需要为此改动。第四，把 AI 的改进拆成可归因的两步：先只重写评估函数（对旧版 7 胜 2 负 11 平），再把搜索换成迭代加深 α-β（对旧版 20 局全胜、自对弈平均回合从 96.9 降到 52.3），从而分别说明「估值」与「深度」各自值多少。

本系统仍有明显的不足。其一，AI 的搜索还算不上快：边长 7 的中局算到第 7 层需要 1.2 秒，没有置换表，也没有增量估值，评估函数每次都要从零遍历全场，因此在低配设备与联机三方观战的场景下会显得迟钝；其二，平衡性数据还不够充分，目前只做了 AI 互搏、AI 对随机与新旧版本对抗，没有引入人类对局的统计，结论的置信度有限；从购买分布看，坦克的性价比明显偏低，说明数值仍需打磨。其三，自建兵种的归属依赖客户端保存的删除凭据，一旦清理浏览器数据就无法再删除自己上传的内容，缺少账号体系是这一问题的根源。其四，功能上尚缺观战、回放与匹配机制，联机依赖房号手工传递，规模化体验不足。

后续可以从三个方向改进：一是给搜索补上置换表与增量估值（局面哈希 + 只重算受影响的格子），把 1.2 秒的预算压到 0.3 秒以内，并在评估函数中引入阵型与威胁评估；二是引入账号体系与基于 Elo 的 AI 参数自校准，用真实对局数据替代经验设定默认值；三是补齐观战与回放（借助已有的序列化能力，回放只需存储初始局面与动作序列），并尝试用云端房间列表替代手工房号。

## 参考文献

[1] Patel A. Hexagonal Grids[EB/OL]. Red Blob Games, 2013[2026-11-02]. https://www.redblobgames.com/grids/hexagons/.

[2] Campbell M, Hoane A J, Hsu F H. Deep Blue[J]. Artificial Intelligence, 2002, 134(1/2): 57-83.

[3] Browne C B, Powley E, Whitehouse D, et al. A survey of Monte Carlo tree search methods[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(1): 1-43.

[4] Silver D, Hubert T, Schrittwieser J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. Science, 2018, 362(6419): 1140-1144.

[5] Fette I, Melnikov A. The WebSocket Protocol: RFC 6455[S]. Internet Engineering Task Force, 2011.

[6] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2005.

[7] Yannakakis G N, Togelius J. Artificial Intelligence and Games[M]. Cham: Springer, 2018.

[8] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. Irvine: University of California, 2000.